JSON Web Token Cheat Sheet

Introduction

This cheat sheet provides tips to prevent common security issues when using JSON Web Tokens (JWT).

JSON Web Tokens (JWT) are security tokens for carrying information (**claims**), often about a user, an application, etc. (**subject**). JWTs can provide authenticity of the claims (**signed JWT**) and/or confidentiality of the claims (**encrypted JWT**). In addition, JWT defines standard claims.

JWTs are used in a wide range of applications such as:

- In OpenID Connect, the ID token is a JWT used to represent the identity and attributes of the connected user.
- In OAuth 2, the access token used to obtain access to a protected resource can be a JWT.
- A JWT is often suggested for “stateless” user sessions. However, this usage is frowned upon.
- A DPoP Proof JWT can be used to prove possession of a private key.
- In SPIFFE, a JWT-SVID can be used to authenticate a workload.

In its most common form (signed JWT), this information is protected by the generating application (**issuer**) using a signature to ensure it has not been tampered with. This signature prevents attackers, such as a malicious client or user, from forging a token or modifying the claims in an existing token, for example changing the user role from a simple user to an admin or altering the client's login. The JWT can be seen as a protected identity card or certificate about a user, an application, etc. An application (**presenter**) presents the token to a consuming application (**audience**) which can verify the token's authenticity and validity and take decisions or actions based on these claims.

JWT can also provide confidentiality of the claims (encrypted JWT). Encryption is currently not treated in this cheat sheet but many aspects of this cheat sheet are applicable to encrypted JWTs.

Token Structure

Signed JWTs have the following structure:

```
{base64url(json(header))}.{base64url(json(claims))}.{base64url(signature)}
```

The following elements are present in signed JWTs:

- **Protected Header:** the JWT header contains some information about the token such as the type of token (IANA media type) and the cryptographic algorithms used to protect the token.
- **Claims:** the content JWT is a list of claims (usually about the subject). See the JWT IANA Registry for a list of standard claims.
- **Signature:** a signature in JWT is either a public-key digital signature (using a public/private key pair) or a MAC (using a shared secret). The signature protects both the protected headers and the claims.

Example

For example, the following example (taken from JWT.IO):

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiYWRtaW4iOnRydWUsImlhdCI6MTU
xNjIzOTAyMn0.KMUFsIDTnFmyG3nMiGM6H9FNFUROf3wh7SmqJp-QV30
```

The first part (**protected header**) can be decoded into:

```
{
  "alg": "HS256",
```

```
  "typ": "JWT"
}
```

The second part (**claims**) can be decoded into:

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true,
  "iat": 1516239022
}
```

The last part (**signature**) guarantees the authenticity of both the header and the claims, either using a public/private key pair (digital signature) or a shared secret (MAC), depending on the `alg` header value. For our example, it is computed as:

```
base64url(
    HMACSHA256(
        "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9"
        + "."
        + "eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiYWRtaW4iOnRydWUsImlhdCI6MTUxNjIzOTAyMn0",
        key
    )
)
```

Considerations about using JWTs

Not using JWTs

Before using JWTs to solve your problems, you should consider if they are really necessary for your use case.

JWTs are often suggested for “stateless” user sessions. However, if you use JWTs for user sessions, you will need a solution for managing session invalidation. This can be achieved using a deny list of revoked sessions/tokens. If your application implements such a deny list,

user sessions won't be completely stateless anymore which might defeat the benefits of stateless sessions. You might want to consider using a plain session system and follow the advices from the dedicated session management cheat sheet.

Public-key Signatures vs. MAC

A signed JWT can be authenticated using either a digital signature or a MAC:

- **When using a digital signature,** the issuer of the token uses its private key to generate a signature. The audience of the token can use the associated public key to verify the authenticity of the token. Whereas the private key must only be known by the issuer, the public key can be public.
- **When using a MAC,** a shared secret is shared between the issuer and the audience. The *same* shared secret is used by the issuer to generate the token and by the audience to verify the authenticity of the token.

The two approaches differ on how credentials are managed.

When using a digital signature:

- The issuer can reuse the same public key for many different audiences.
- The audience of the token only need public information to validate the token authenticity which removes the risk of secret leakage by the audience.
- Because the public key does not need to be secret, it can easily be distributed (eg. by publishing it at a public HTTPS URI).
- This makes key rotation simpler as well.
- Traditional digital signature schemes might be broken by post quantum computers in the future. They would need to be replaced with post-quantum digital signature schemes which heavier are traditional signature schemes.

When using a MAC:

- A different secret must be used for each (issuer, audience) pair. If, for example, the same secret is reused for difference audiences, one audience can forge a token (impersonating the issuer).

- Secret keys must obviously not be published at a public HTTPS URI. Some solution for secret distribution and rotation must be found.
- MAC are much faster than digital signatures (but this is usually negligible in practice).

Using a MAC may be interesting in the following cases:

- The issuer of the token is the sole audience of the token. Even in this case, it might be easier to use digital signature for secret rotation/distribution.
- The issuer of the token is the audience. In this case, there is no problem of secret rotation/distribution.

Public-key Signatures

Signature scheme	Identifier	Type	Status
EdDSA	EdDSA, Ed448, Ed25519	Traditional	Recommended, limited support
ECDSA	ES256, ES384, ES512	Traditional	Recommended
RSASSA-PSS	PS256, PS384, PS512	Traditional	Recommended
RSASSA-PKCS1-v1_5	RS256, RS384, RS512	Traditional	Not recommended
Hybrid ML-DSA / EdDSA	ML-DSA-44-Ed25519, etc.	PQ/T hybrid	Draft
Hybrid ML-DSA / ECDSA	ML-DSA-44-ES256, etc.	PQ/T hybrid	Draft
ML-DSA	ML-DSA-44, ML-DSA-65, ML-DSA-87	Post-quantum	Very limited support at best

Explanations:

- Support for EdDSA in JWT implementations is currently limited.
- Generating ECDSA signatures may be dangerous on embedded systems where the quality of the randomness may be problematic. In this case, you the implementation must use deterministic ECDA as defined in RFC 6979.
- Post-quantum signatures (ML-DSA) or hybrid post quantum signatures are designed to be resistant against quantum computers. However, they produce very large signatures, resulting in very large JWTs. Their usage is probably not justified at the moment unless you need signatures with a long validity.

Key management:

- Do not reuse the key pair for another purpose (eg. for encryption).
- Using the same key for authenticating different types of JWTs is fine as long as this does not introduce a risk of token type confusion.
- Do not publish your private key!

MAC

Signature scheme	Identifier	Status
HMAC with SHA-2	HS256, HS384, HS512	Recommended

Secret management:

- Do not reuse the same secret for another purpose (eg. for encryption).
- Using the same key for authenticating different types of JWTs is fine as long as this does not introduce a risk of token type confusion.
- Do not reuse the same secret with another audience.
- Do not reuse the same secret with another issuer.

- Do not use a password as MAC secret.
- The secret must be generated using a local, cryptographically secure secret generator.
- The secret must have at least the same size as the output (eg. 256, 384 and 512 bits respectively for HS256, HS384 and HS512).
- Do not publish your secret key!
- The secret must have at least 160 bits of entropy.
- For HMAC, the secret should be at least as long as the output size.

Valid HMAC secret generation example:

```
import secrets
secret_for_hs256 = secrets.token_bytes(256//8)
secret_for_hs512 = secrets.token_bytes(512//8)
```

Invalid HMAC secret generation:

```
import random
import secrets

# Using a password/passphrase is not OK:
bad_secret = b"MyProject2026"

# Using a hardcoded secret is not OK:
bad_secret = urlsafe_b64decode(b'KYkbbclxtjJMiHzoPvuahOfarej0VV-nQZPFxK0hyro=')

# Not a secure randomness source:
bad_secret = random.randbytes(256//8)

# Not enough entropy:
bad_secret = secrets.token_bytes(128//8)

# Not enough entropy for HS512
meh_secret_for_hs512 = secrets.token_bytes(256//8)
```

Issues

None Hashing Algorithm

Symptom

This attack, described here, occurs when an attacker alters the token and changes the hashing algorithm to indicate, through the *none* keyword, that the integrity of the token has already been verified. As explained in the link above *some libraries treated tokens signed with the none algorithm as a valid token with a verified signature*, so an attacker can alter the token claims and the modified token will still be trusted by the application.

How to Prevent

First, use a JWT library that is not exposed to this vulnerability.

Last, during token validation, explicitly request that the expected algorithm was used.

Implementation Example

```
// HMAC key - Block serialization and storage as String in JVM memory
private transient byte[] keyHMAC = ...;

...

//Create a verification context for the token requesting
//explicitly the use of the HMAC-256 hashing algorithm
JWTVerifier verifier = JWT.require(Algorithm.HMAC256(keyHMAC)).build();

//Verify the token, if the verification fail then a exception is thrown
DecodedJWT decodedToken = verifier.verify(token);
```

Token Sidejacking

Symptom

This attack occurs when a token has been intercepted/stolen by an attacker and they use it to gain access to the system using targeted user identity.

How to Prevent

One way to prevent this is by adding a "user context" to the token. The user context should consist of the following:

- A random string generated during the authentication phase. This string is sent to the client as a hardened cookie (with the following flags: HttpOnly + Secure, SameSite, Max-Age, and cookie prefixes). Avoid setting the *expires* header so the cookie is cleared when the browser is closed. Set *Max-Age* to a value equal to or less than the JWT's expiry time — never more.
- A SHA256 hash of the random string will be stored in the token (instead of the raw value) in order to prevent any XSS issues allowing the attacker to read the random string value and setting the expected cookie.

Avoid using IP addresses as part of the context. IP addresses can change during a single session due to legitimate reasons — for example, when a user accesses the application on a mobile device and switches network providers. Additionally, IP tracking can raise concerns related to GDPR compliance in the EU.

During token validation, if the received token does not contain the correct context (e.g., if it is being replayed by an attacker), it must be rejected.

Implementation example

Code to create the token after successful authentication.

```
// HMAC key - Block serialization and storage as String in JVM memory
private transient byte[] keyHMAC = ...;
// Random data generator
private SecureRandom secureRandom = new SecureRandom();

...
```

```
//Generate a random string that will constitute the fingerprint for this user
byte[] randomFgp = new byte[50];
secureRandom.nextBytes(randomFgp);
String userFingerprint = DatatypeConverter.printHexBinary(randomFgp);

//Add the fingerprint in a hardened cookie - Add cookie manually because
//SameSite attribute is not supported by javax.servlet.http.Cookie class
String fingerprintCookie = "__Secure-Fgp=" + userFingerprint
                         + "; SameSite=Strict; HttpOnly; Secure";
response.addHeader("Set-Cookie", fingerprintCookie);

//Compute a SHA256 hash of the fingerprint in order to store the
//fingerprint hash (instead of the raw value) in the token
//to prevent an XSS to be able to read the fingerprint and
//set the expected cookie itself
MessageDigest digest = MessageDigest.getInstance("SHA-256");
byte[] userFingerprintDigest = digest.digest(userFingerprint.getBytes("utf-8"));
String userFingerprintHash = DatatypeConverter.printHexBinary(userFingerprintDigest);

//Create the token with a validity of 15 minutes and client context (fingerprint) information
Calendar c = Calendar.getInstance();
Date now = c.getTime();
c.add(Calendar.MINUTE, 15);
Date expirationDate = c.getTime();
Map<String, Object> headerClaims = new HashMap<>();
headerClaims.put("typ", "JWT");
String token = JWT.create().withSubject(login)
   .withExpiresAt(expirationDate)
   .withIssuer(this.issuerID)
   .withIssuedAt(now)
   .withNotBefore(now)
   .withClaim("userFingerprint", userFingerprintHash)
   .withHeader(headerClaims)
   .sign(Algorithm.HMAC256(this.keyHMAC));
```

Code to validate the token.

```
// HMAC key - Block serialization and storage as String in JVM memory
private transient byte[] keyHMAC = ...;

...

//Retrieve the user fingerprint from the dedicated cookie
String userFingerprint = null;
if (request.getCookies() != null && request.getCookies().length > 0) {
 List<Cookie> cookies = Arrays.stream(request.getCookies()).collect(Collectors.toList());
 Optional<Cookie> cookie = cookies.stream().filter(c -> "__Secure-Fgp"
                                          .equals(c.getName())).findFirst();
 if (cookie.isPresent()) {
   userFingerprint = cookie.get().getValue();
 }
}

//Compute a SHA256 hash of the received fingerprint in cookie in order to compare
//it to the fingerprint hash stored in the token
MessageDigest digest = MessageDigest.getInstance("SHA-256");
byte[] userFingerprintDigest = digest.digest(userFingerprint.getBytes("utf-8"));
String userFingerprintHash = DatatypeConverter.printHexBinary(userFingerprintDigest);

//Create a verification context for the token
JWTVerifier verifier = JWT.require(Algorithm.HMAC256(keyHMAC))
                          .withIssuer(issuerID)
                          .withClaim("userFingerprint", userFingerprintHash)
                          .build();

//Verify the token, if the verification fail then an exception is thrown
DecodedJWT decodedToken = verifier.verify(token);
```

No Built-In Token Revocation by the User

Symptom

This problem is inherent to JWT because a token only becomes invalid when it expires. The user has no built-in feature to explicitly revoke the validity of a token. This means that if it is stolen, a user cannot revoke the token itself thereby blocking the attacker.

How to Prevent

Since JWTs are stateless, There is no session maintained on the server(s) serving client requests. As such, there is no session to invalidate on the server side. A well implemented Token Sidejacking solution (as explained above) should alleviate the need for maintaining denylist on server side. This is because a hardened cookie used in the Token Sidejacking can be considered as secure as a session ID used in the traditional session system, and unless both the cookie and the JWT are intercepted/stolen, the JWT is unusable. A logout can thus be 'simulated' by clearing the JWT from session storage. If the user chooses to close the browser instead, then both the cookie and sessionStorage are cleared automatically.

Another way to protect against this is to implement a token denylist that will be used to mimic the "logout" feature that exists with traditional session management system.

When the user wants to "logout" then it call a dedicated service that will add the token's identifying claims (`jti` and `iss`) to the denylist resulting in an immediate invalidation of the token for further usage in the application.

Note:

Do not use the raw JWT or a hash of it as the denylist key.

A denylist keyed on a digest of the raw token (e.g. `SHA-256(token)`) is unsafe, because a JWT does not have a single canonical byte representation. The same logically valid token can be transformed into a *different* byte sequence that still passes signature verification — which means it hashes differently and silently bypasses the denylist. This can happen for two independent reasons:

- **ECDSA signature malleability.** For JWTs signed with an ECDSA algorithm (e.g. `ES256`), a valid signature `(r, s)` has a second, equally valid form `(r, (-s) mod n)` , where `n` is the order of the curve's generator point. Both signatures verify successfully against the same public key for the same header and payload, but produce different token bytes — and therefore a different digest. An attacker in possession of a revoked token can compute this alternate signature and obtain a token that still authenticates.

- **Non-strict JWT parsing.** Many JWT libraries tolerate multiple, non-canonical encodings of the same logical token — for example, base64url values with extraneous padding, alternate-but-decodable character substitutions, or trailing bytes with differing unused bits that decode to identical content. These variants are byte-for-byte different from the original token and therefore also bypass a hash-based denylist, regardless of signing algorithm (HMAC, RSA, or ECDSA).

Because of this, the denylist must be keyed on a value that is **stable across these malleable encodings**, not on the token's raw bytes. The recommended approach is to use the `jti` (JWT ID) claim, which is a unique identifier assigned by the issuer at creation time and embedded inside the signed payload — making it immune to the malleability classes above, since any tampering with it invalidates the signature. Combining `jti` with the `iss` (issuer) claim ensures a globally unique denylist key — `jti` uniqueness is only guaranteed within a single issuer, so a (`jti`, `iss`) pair is required to prevent collisions between tokens from different issuers. Note that this denylist is audience-maintained (operated by the relying party), not by the issuer itself.

Implementation Example

BLOCK LIST STORAGE

The following example demonstrates a denylist keyed on `jti` and `iss` rather than the raw token, per the recommendation above.

A database table with the following structure will be used as the central denylist storage.

```
create table if not exists revoked_token(
  jwt_id varchar(255) not null,
  iss varchar(255) not null,
  expires_at timestamp not null,
  revocation_date timestamp default now(),
  primary key (jwt_id, iss)
);
```

TOKEN REVOCATION MANAGEMENT

Code in charge of adding a token to the denylist and checking if a token is revoked.

```
/**
 * Handle the revocation of the token (logout).
 * Revocation is keyed on the token's "jti" and "iss" claims rather than
 * a hash of the raw token, since JWTs do not have a single canonical
 * byte representation (see warning above) and a raw-token or
 * digest-based denylist can be bypassed via ECDSA signature
 * malleability or lenient JWT parsing. Both claims are embedded in the
 * signed payload, so neither can be altered without invalidating the
 * signature. The (jti, iss) pair is used because jti uniqueness is
 * only guaranteed per issuer — a malicious or rogue issuer could mint
 * a JWT with the same jti as a legitimate one, causing a collision.
 * Note: this denylist is audience-maintained (operated by the relying
 * party), not by the issuer itself.
 * Use a DB in order to allow multiple instances to check for revoked
 * tokens and allow cleanup at centralized DB level.
 */
public class TokenRevoker {

    /** DB Connection */
    @Resource("jdbc/storeDS")
    private DataSource storeDS;

    /**
     * Verify if a given token (identified by its "jti" + "iss" claims)
     * is present in the revocation table.
     *
     * @param decodedToken Verified, decoded token (signature already validated)
     * @return Presence flag
     * @throws Exception If any issue occurs during communication with DB
     */
    public boolean isTokenRevoked(DecodedJWT decodedToken) throws Exception {
        String jwtId = decodedToken.getId();       // value of the "jti" claim
        String issuer = decodedToken.getIssuer();  // value of the "iss" claim

        if (jwtId == null || jwtId.trim().isEmpty() || issuer == null || issuer.trim().isEmpty()) {
            // A token without "jti" or "iss" cannot be safely tracked
            // in this denylist; such a token should be rejected upstream.
            throw new IllegalArgumentException("Token has no \"jti\" or \"iss\" claim");
```

```
        }

        boolean tokenIsPresent;
        try (Connection con = this.storeDS.getConnection()) {
            String query = "select jwt_id from revoked_token where jwt_id = ? and iss = ?";
            try (PreparedStatement pStatement = con.prepareStatement(query)) {
                pStatement.setString(1, jwtId);
                pStatement.setString(2, issuer);
                try (ResultSet rSet = pStatement.executeQuery()) {
                    tokenIsPresent = rSet.next();
                }
            }
        }
        return tokenIsPresent;
    }

    /**
     * Add a token's "jti" + "iss" claims to the revocation table, along
     * with the token's expiration so the entry can be purged once it is
     * no longer needed (i.e. once the token itself would have expired
     * naturally).
     *
     * @param decodedToken Verified, decoded token (signature already validated)
     * @throws Exception If any issue occurs during communication with DB
     */
    public void revokeToken(DecodedJWT decodedToken) throws Exception {
        String jwtId = decodedToken.getId();
        String issuer = decodedToken.getIssuer();
        Date expiresAt = decodedToken.getExpiresAt(); // value of the "exp" claim

        if (jwtId == null || jwtId.trim().isEmpty() || issuer == null || issuer.trim().isEmpty()) {
            throw new IllegalArgumentException("Token has no \"jti\" or \"iss\" claim");
        }
        if (expiresAt == null) {
            throw new IllegalArgumentException("Token has no \"exp\" claim; cannot schedule purge");
        }

        if (!this.isTokenRevoked(decodedToken)) {
```

```
            try (Connection con = this.storeDS.getConnection()) {
                String query = "insert into revoked_token(jwt_id, iss, expires_at) values(?, ?, ?)";
                int insertedRecordCount;
                try (PreparedStatement pStatement = con.prepareStatement(query)) {
                    pStatement.setString(1, jwtId);
                    pStatement.setString(2, issuer);
                    pStatement.setTimestamp(3, new java.sql.Timestamp(expiresAt.getTime()));
                    insertedRecordCount = pStatement.executeUpdate();
                }
                if (insertedRecordCount != 1) {
                    throw new IllegalStateException("Number of inserted record is invalid," +
                    " 1 expected but is " + insertedRecordCount);
                }
            }
        }
    }

    /**
     * Purge expired entries from the denylist. Intended to be run on a
     * schedule (e.g. a daily cron job or scheduled task) so the table
     * does not grow unbounded -- once a token's own "exp" has passed,
     * it would already be rejected by signature/expiry validation, so
     * keeping its denylist entry is no longer necessary.
     *
     * @throws Exception If any issue occurs during communication with DB
     */
    public void purgeExpiredEntries() throws Exception {
        try (Connection con = this.storeDS.getConnection()) {
            String query = "delete from revoked_token where expires_at < ?";
            try (PreparedStatement pStatement = con.prepareStatement(query)) {
                pStatement.setTimestamp(1, new java.sql.Timestamp(System.currentTimeMillis()));
                pStatement.executeUpdate();
            }
        }
    }
}
```

Issuer-Side Revocation: Token Status List

The denylist approach described above is typically operated by the relying party (resource server) — it works well when the audience maintains its own revocation state close to where tokens are validated. However, in some deployments the issuer needs to centrally broadcast revocation state to multiple relying parties without requiring each one to maintain its own denylist.

For this use case, the IETF Token Status List (TSL) draft defines a scalable, issuer-maintained revocation mechanism. TSL is used in SD-JWT Verifiable Credentials and other high-scale deployments where a single issuer serves many relying parties. Consult the TSL draft for implementation guidance when issuer-side revocation is required.

Token Information Disclosure

Symptom

This attack occurs when an attacker has access to a token (or a set of tokens) and extracts information stored in it (the contents of JWTs are base64 encoded, but is not encrypted by default) in order to obtain information about the system. Information can be for example the security roles, login format...

How to Prevent

A way to protect against this attack is to cipher the token using, for example, a symmetric algorithm.

It's also important to protect the ciphered data against attack like Padding Oracle or any other attack using cryptanalysis.

In order to achieve all these goals, the *AES-GCM* algorithm is used which provides *Authenticated Encryption with Associated Data*.

More details from here:

```
AEAD primitive (Authenticated Encryption with Associated Data) provides functionality of symmetric
authenticated encryption.

Implementations of this primitive are secure against adaptive chosen ciphertext attacks.

When encrypting a plaintext one can optionally provide associated data that should be authenticated
but not encrypted.
```

```
That is, the encryption with associated data ensures authenticity (ie. who the sender is) and
integrity (ie. data has not been tampered with) of that data, but not its secrecy.

See RFC5116: https://tools.ietf.org/html/rfc5116
```

Note:

Here ciphering is added mainly to hide internal information but it's very important to remember that the first protection against tampering of the JWT is the signature. So, the token signature and its verification must be always in place.

Implementation Example

TOKEN CIPHERING

Code in charge of managing the ciphering. Google Tink dedicated crypto library is used to handle ciphering operations in order to use built-in best practices provided by this library.

```
/**
 * Handle ciphering and deciphering of the token using AES-GCM.
 *
 * @see "https://github.com/google/tink/blob/master/docs/JAVA-HOWTO.md"
 */
public class TokenCipher {

    /**
     * Constructor - Register AEAD configuration
     *
     * @throws Exception If any issue occur during AEAD configuration registration
     */
    public TokenCipher() throws Exception {
        AeadConfig.register();
    }

    /**
     * Cipher a JWT
```

```
     *
     * @param jwt          Token to cipher
     * @param keysetHandle Pointer to the keyset handle
     * @return The ciphered version of the token encoded in HEX
     * @throws Exception If any issue occur during token ciphering operation
     */
    public String cipherToken(String jwt, KeysetHandle keysetHandle) throws Exception {
        //Verify parameters
        if (jwt == null || jwt.isEmpty() || keysetHandle == null) {
            throw new IllegalArgumentException("Both parameters must be specified!");
        }

        //Get the primitive
        Aead aead = AeadFactory.getPrimitive(keysetHandle);

        //Cipher the token
        byte[] cipheredToken = aead.encrypt(jwt.getBytes(), null);

        return DatatypeConverter.printHexBinary(cipheredToken);
    }

    /**
     * Decipher a JWT
     *
     * @param jwtInHex     Token to decipher encoded in HEX
     * @param keysetHandle Pointer to the keyset handle
     * @return The token in clear text
     * @throws Exception If any issue occur during token deciphering operation
     */
    public String decipherToken(String jwtInHex, KeysetHandle keysetHandle) throws Exception {
        //Verify parameters
        if (jwtInHex == null || jwtInHex.isEmpty() || keysetHandle == null) {
            throw new IllegalArgumentException("Both parameters must be specified !");
        }

        //Decode the ciphered token
        byte[] cipheredToken = DatatypeConverter.parseHexBinary(jwtInHex);
```

```
        //Get the primitive
        Aead aead = AeadFactory.getPrimitive(keysetHandle);

        //Decipher the token
        byte[] decipheredToken = aead.decrypt(cipheredToken, null);

        return new String(decipheredToken);
    }
}
```

CREATION / VALIDATION OF THE TOKEN

Use the token ciphering handler during the creation and the validation of the token.

Load keys (ciphering key was generated and stored using Google Tink) and setup cipher.

```
//Load keys from configuration text/json files in order to avoid to storing keys as a String in JVM memory
private transient byte[] keyHMAC = Files.readAllBytes(Paths.get("src", "main", "conf", "key-hmac.txt"));
private transient KeysetHandle keyCiphering = CleartextKeysetHandle.read(JsonKeysetReader.withFile(
Paths.get("src", "main", "conf", "key-ciphering.json").toFile()));

...

//Init token ciphering handler
TokenCipher tokenCipher = new TokenCipher();
```

Token creation.

```
//Generate the JWT token using the JWT API...
//Cipher the token (String JSON representation)
String cipheredToken = tokenCipher.cipherToken(token, this.keyCiphering);
//Send the ciphered token encoded in HEX to the client in HTTP response...
```

Token validation.

```
//Retrieve the ciphered token encoded in HEX from the HTTP request...
//Decipher the token
String token = tokenCipher.decipherToken(cipheredToken, this.keyCiphering);
//Verify the token using the JWT API...
//Verify access...
```

Token Storage on Client Side

Symptom

This occurs when an application stores the token in a manner exhibiting the following behavior:

- Automatically sent by the browser (*Cookie* storage).
- Retrieved even if the browser is restarted (Use of browser *localStorage* container).
- Retrieved in case of XSS issue (Cookie accessible to JavaScript code or Token stored in browser local/session storage).

How to Prevent

1. Store the token using the browser *sessionStorage* container, or use JavaScript *closures* with *private* variables
2. Add it as a *Bearer* HTTP `Authentication` header with JavaScript when calling services.
3. Add fingerprint information to the token.

By storing the token in browser *sessionStorage* container it exposes the token to being stolen through an XSS attack. However, fingerprints added to the token prevent reuse of the stolen token by the attacker on their machine. To close a maximum of exploitation surfaces for an attacker, add a browser Content Security Policy to harden the execution context.

But, we know that *sessionStorage* is not always practical due to its per-tab scope, and the storage method for tokens should balance *security* and *usability*.

LocalStorage is a better method than *sessionStorage* for usability because it allows the session to persist between browser restarts and across tabs, but you must use strict security controls:

- Tokens stored in *localStorage* should have *short expiration times* (e.g., *15-30 minutes idle timeout, 8-hour absolute timeout*).
- Implement mechanisms such as *token rotation* and *refresh tokens* to minimize risk.

If *session persistence across tabs* and *sessionStorage* are required, consider using *BroadcastChannel API* or *Single Sign-On (SSO)* to re-authenticate users automatically when they open new tabs.

An alternative to storing token in browser *sessionStorage* or in *localStorage* is to use JavaScript private variable or Closures. In this, access to all web requests are routed through a JavaScript module that encapsulates the token in a private variable which can not be accessed other than from within the module.

Note:

- The remaining case is when an attacker uses the user's browsing context as a proxy to use the target application through the legitimate user but the Content Security Policy can prevent communication with non expected domains.
- It's also possible to implement the authentication service in a way that the token is issued within a hardened cookie, but in this case, protection against a Cross-Site Request Forgery attack must be implemented.

Implementation Example

JavaScript code to store the token after authentication.

```
/* Handle request for JWT token and local storage*/
function authenticate() {
    const login = $("#login").val();
    const postData = "login=" + encodeURIComponent(login) + "&password=test";

    $.post("/services/authenticate", postData, function (data) {
        if (data.status == "Authentication successful!") {
            ...
            sessionStorage.setItem("token", data.token);
```

```
        }
        else {
            ...
            sessionStorage.removeItem("token");
        }
    })
    .fail(function (jqXHR, textStatus, error) {
        ...
        sessionStorage.removeItem("token");
    });
}
```

JavaScript code to add the token as a *Bearer* HTTP Authentication header when calling a service, for example a service to validate token here.

```
/* Handle request for JWT token validation */
function validateToken() {
    var token = sessionStorage.getItem("token");

    if (token == undefined || token == "") {
        $("#infoZone").removeClass();
        $("#infoZone").addClass("alert alert-warning");
        $("#infoZone").text("Obtain a JWT token first :)");
        return;
    }

    $.ajax({
        url: "/services/validate",
        type: "POST",
        beforeSend: function (xhr) {
            xhr.setRequestHeader("Authorization", "bearer " + token);
        },
        success: function (data) {
            ...
        },
        error: function (jqXHR, textStatus, error) {
            ...
```

```
        },
    });
}
```

JavaScript code to implement closures with private variables:

```
function myFetchModule() {
    // Protect the original 'fetch' from getting overwritten via XSS
    const fetch = window.fetch;

    const authOrigins = ["https://yourorigin", "http://localhost"];
    let token = '';

    this.setToken = (value) => {
        token = value
    }

    this.fetch = (resource, options) => {
        let req = new Request(resource, options);
        destOrigin = new URL(req.url).origin;
        if (token && authOrigins.includes(destOrigin)) {
            req.headers.set('Authorization', token);
        }
        return fetch(req)
    }
}

...

// usage:
const myFetch = new myFetchModule()

function login() {
  fetch("/api/login")
      .then((res) => {
          if (res.status == 200) {
              return res.json()
```

```
            } else {
                throw Error(res.statusText)
            }
        })
        .then(data => {
            myFetch.setToken(data.token)
            console.log("Token received and stored.")
        })
        .catch(console.error)
}

...

// after login, subsequent api calls:
function makeRequest() {
    myFetch.fetch("/api/hello", {headers: {"MyHeader": "foobar"}})
        .then((res) => {
            if (res.status == 200) {
                return res.text()
            } else {
                throw Error(res.statusText)
            }
        }).then(responseText => console.log("helloResponse", responseText))
        .catch(console.error)
}
```

Weak Token Secret

Symptom

When the token is protected using an HMAC based algorithm, the security of the token is entirely dependent on the strength of the secret used with the HMAC. If an attacker can obtain a valid JWT, they can then carry out an offline attack and attempt to crack the secret using tools such as John the Ripper or Hashcat.

If they are successful, they would then be able to modify the token and re-sign it with the key they had obtained. This could let them escalate their privileges, compromise other users' accounts, or perform other actions depending on the contents of the JWT.

There are a number of guides that document this process in greater detail.

How to Prevent

The simplest way to prevent this attack is to ensure that the secret used to sign the JWTs is strong and unique, in order to make it harder for an attacker to crack. As this secret would never need to be typed by a human, it should be at least 64 characters, and generated using a secure source of randomness.

Alternatively, consider the use of tokens that are signed using a digital signature (public-key cryptography) rather than using an HMAC and secret key.

Relation to other formats

JWT is a profile of the more general JOSE format (RFC 7515, RFC 7516). While this cheat sheet is focused on JWTs, a large part of what is discussed here is more generally applicable to JOSE messages in general.

Conversely, CWT, and more generally COSE, have a very similar design and many of the things discussed might be applicable to CWT and COSE as well.

Depending on the application, some alternatives to JWT and JOSE might be:

- opaque tokens;
- CBOR Object Token (CWT) and CBOR Object Signing and Encryption (COSE);
- PASETO;
- Eclipse Biscuit;
- Fernet;

- Security Assertion Markup Language (SAML) and XML signature.

References

Main JWT and JOSE specifications:

- RFC 7515, JSON Web Signature (JWS)
- RFC 7516, JSON Web Encryption (JWE)
- RFC 7517, JSON Web Key (JWK)
- RFC 7519, JSON Web Token (JWT)
- RFC 8725, JWT Best Practices

Some applications of JWTs:

- JWT Profile for OAuth 2.0 Access Tokens

IANA registries:

- JSON Object Signing and Encryption (JOSE) IANA Reguistry
- JSON Web Token IANA Reguistry (JWT)

Attacks on JWT and JOSE:

- {JWT}.{Attack}.Playbook - A project documents the known attacks and potential security vulnerabilities and misconfigurations of JSON Web Tokens.
- JWT.io Discussion Forum (Hosted by Auth0)

Other useful links:

- JWT IANA Registry